

Agentic AI Testing & AI Quality Engineering

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This syllabus lists every module of the course in teaching order, with the topics covered in each. It is generated from the course page, so the two cannot disagree.

Module 1: Agentic AI Architecture for Testers

Schedule: Week 1

What an agent is, defined by what it can do rather than by marketing. An agent versus a chatbot, and an agent versus a deterministic workflow — the second distinction matters most, because a great deal of what is sold as agentic is a scripted pipeline with a model in one step, and it should be tested as a pipeline.

Then the reasoning loop: plan, act, observe, repeat. Tools and function calling, state, memory, human-in-the-loop checkpoints and multi-agent topologies, each introduced with the question a tester should ask about it.

Topics

- What an AI agent is
- Agent versus chatbot
- Agent versus deterministic workflow with a model in it
- The reasoning loop — plan, act, observe
- Tools and function calling
- State and how it accumulates
- Short-term and long-term memory
- Human-in-the-loop checkpoints
- Multi-agent systems in outline
- Agent frameworks at a conceptual level

Module 2: Agent Test Strategy

Schedule: Week 1

The decision that shapes every suite you will write: which parts of this system are deterministic and must be asserted, and which are probabilistic and must be evaluated. Tools are code and get unit tests. Tool selection is a judgement and gets evaluation. Confusing the two produces either a flaky suite or one that tests nothing.

From there: the test pyramid for agents, golden tasks with defined success criteria, rubrics, and the environment question — mock tools, sandboxed execution, and why an agent test that touches a real system is a test you will eventually regret.

Topics

- What must be deterministic, and what must be evaluated
- Unit testing tools and functions
- Integration testing the agent against its tools
- End-to-end agent tests and their cost
- Scenario-based evaluation
- Golden tasks and success criteria
- Rubrics for partial success
- Test environments for agents
- Mock tools versus sandboxed real tools
- Why agent tests should not touch production systems

Module 3: Tool-Calling Testing

Schedule: Week 2

The highest-value test surface in the whole course, because this is where an agent's mistakes become actions. Correct tool selection, wrong tool selection, and the underrated no-tool case — an agent that calls a tool when it should simply have answered is a defect, and it is rarely caught.

Then arguments: correct, missing, invalid, and the dangerous middle case of plausible but wrong. Schema validation, tool permissions, tool errors, retry logic, idempotency and timeouts follow, with mocking taught as the mechanism that makes all of it testable.

Topics

- Correct tool selection under ambiguity
- Wrong tool selection and how to detect it
- The no-tool case — answering without acting
- Correct, missing and invalid arguments
- Plausible but wrong arguments
- Argument schema validation
- Tool permissions and allowlists
- Tool errors and how the agent responds
- Retry logic and idempotency
- Timeout behaviour
- External dependency failure
- Building tool mocks that behave badly on purpose

Module 4: Workflow and Trajectory Evaluation**Schedule: Weeks 2–3**

Judging the path. A successful trajectory is not simply one that reached the end state — it took a reasonable number of steps, in a defensible order, without repeating calls or looping, and it did the critical steps rather than skipping to a plausible conclusion.

You learn to specify expected critical steps while permitting alternative valid paths, and to detect the specific pathologies: missing steps, duplicates, repeated identical calls, infinite loops, premature completion and wrong ordering. Recovery behaviour is evaluated too, since an agent that errs and corrects itself may be better than one that never erred by luck.

Topics

- What makes a trajectory successful
- Expected critical steps versus alternative valid paths
- Missing and duplicate steps
- Repeated identical calls
- Infinite loops and step budgets
- Premature completion and false success
- Wrong ordering with a correct end state
- Error recovery as a positive signal
- Step-level versus end-state evaluation
- Writing a trajectory evaluator

Module 5: Memory and Context Testing**Schedule: Week 3**

State is what makes agent bugs irreproducible, and memory is state that persists past the conversation. Session context, conversation history, short-term and long-term memory, and retrieval from memory — each with its failure mode.

Stale memory, incorrect memory and conflicting memories are the quality defects; the correction case matters most in practice, because a user who says "no, my address changed" and is ignored has lost trust permanently. Cross-session

isolation and cross-user leakage are the security defects, and they are the ones that stop a release.

Topics

- Session context and conversation history
- Short-term versus long-term memory
- Memory retrieval and its relevance
- Stale memory
- Incorrect memory
- Conflicting memories
- User correction — and being ignored
- Cross-session isolation
- Cross-user leakage
- PII held in memory
- Deletion and retention behaviour

Module 6: Multi-Agent Testing

Schedule: Week 4

When one agent delegates to another, the failure surface stops being linear. Roles, supervisor and worker patterns, delegation, handoffs and the message contract between agents — which is the interface most worth testing, because it is rarely specified anywhere.

The distributed-systems pathologies arrive with the topology: context lost in transfer, shared state written twice, agents that disagree, duplicated work, deadlocks, and failures that propagate silently rather than surfacing. Escalation to a human is treated as a required path and tested as one.

Topics

- Agent roles and responsibilities
- Supervisor and worker patterns
- Delegation and handoffs
- Message contracts between agents
- Context transfer and what gets lost
- Shared state and write conflicts
- Conflicting agents
- Duplicate work
- Deadlocks and stalls
- Failure propagation
- Escalation to a human as a tested path
- End-to-end task completion across agents

Module 7: Agent Security and Safety

Schedule: Weeks 4–5

The security surface that only exists because the agent can act. Direct prompt injection is the familiar case; indirect injection is the serious one — instructions arriving inside a retrieved document, a web page, a support ticket or a tool's own output, aimed at an agent that has permission to do something about them.

Data exfiltration, secret leakage, unauthorised actions and privilege boundaries follow, then excessive agency: an agent permitted to do more than the task needs. Tool allowlists, confirmation gates for sensitive actions and auditability are the controls you test for, with red-team scenarios as the method.

Topics

- Direct prompt injection
- Indirect injection from documents, web content and tickets

- Malicious tool output
- Untrusted content in the agent's context
- Data exfiltration paths
- Secret leakage
- Unauthorised actions
- Privilege boundaries between users and agents
- Tool allowlists
- Confirmation gates for irreversible actions
- Excessive agency — permission beyond the task
- Unsafe loops and runaway cost
- Auditability and what a trace must record
- Red-team scenarios for action-taking agents

Module 8: Agent Evaluation Metrics

Schedule: Week 5

What you report, and what each number hides. Task success rate is the headline and the most misleading in isolation — an agent succeeding 90% of the time at four times the cost of the alternative is not obviously better.

Tool-selection and argument accuracy, step count, trajectory quality, error-recovery rate, escalation quality and final-response quality give the shape of behaviour; latency, token consumption and cost per completed task give the economics; and reliability across repeated runs gives the confidence interval, because a single run of a probabilistic system is an anecdote.

Topics

- Task success rate, and what it conceals
- Tool-selection accuracy
- Argument accuracy
- Step count and efficiency
- Trajectory quality scoring
- Error-recovery rate
- Human-escalation quality
- Final-response quality
- Safety adherence rate
- Latency and token consumption
- Cost per completed task
- Reliability across repeated runs

Module 9: Offline Evaluation

Schedule: Week 6

The repeatable suite that gates a release. Golden task datasets, simulation, mock tools and deterministic fixtures — enough control that a run difference means the agent changed rather than the world did.

Repeat runs and baselines make the results comparable; model, prompt and agent-version comparison make them actionable. Release gates close the module: a threshold that blocks a deploy, which is where all of this stops being analysis and becomes quality engineering.

Topics

- Golden task datasets for agents
- Simulation and scripted environments
- Mock tools and deterministic fixtures
- Repeat runs and variance
- Baselines and stored results

- Model comparison
- Prompt comparison
- Agent-version comparison
- Release gates and thresholds

Module 10: Online and Production Quality Engineering

Schedule: Week 6

Offline evaluation cannot cover the input distribution of real users, so production becomes a test surface. Tracing, sampling, online evaluators and user feedback, with failed-task capture feeding the regression dataset — the loop that makes a suite get better instead of going stale.

Drift and change detection, incident review, cost and latency monitoring, quality dashboards and human review queues complete the picture. This is the module that turns a tester into a quality engineer, and it is the part most teams have not built.

Topics

- Tracing an agent run in production
- Sampling strategies
- Online evaluators
- User feedback signals and their bias
- Failed-task capture
- Growing the regression dataset from production
- Drift and change detection
- Incident review for agent failures
- Cost and latency monitoring
- Quality dashboards
- Human review queues

Module 11: Test Automation for Agents

Schedule: Week 7

Building the harness. A Python agent-test framework with pytest scenarios, tool mocks, structured trace capture, and evaluators for the properties earlier modules defined.

Assertions come in two kinds and both are used: hard assertions on tool calls and final state, and model judges for the soft criteria that cannot be asserted. Result storage, quality thresholds and CI execution make it a release gate rather than a script someone runs when they remember to.

Topics

- Python agent-test harness structure
- pytest scenarios for agent tasks
- Tool mocks and controllable failure injection
- Capturing structured traces
- Writing evaluators for trajectory and outcome
- Assertions on tool calls
- Assertions on final state
- LLM judges for soft criteria
- CI regression execution
- Test result storage and history
- Quality thresholds that block a deploy

Module 12: Modern Agent Interfaces and the Capstone

Schedule: Weeks 7–8

The interface layer, kept conceptual so it ages well: function and tool-calling patterns, the Model Context Protocol as an example of an agent-to-tool contract, browser and computer-use testing, external connectors, permission boundaries and auditing.

Then the capstone: a full quality-engineering deliverable for an agent — golden task dataset, mocked tools, trajectory evaluator, task-success evaluator, safety tests, CI regression suite and a quality report someone can make a release decision from without reading your code.

Topics

- Function and tool-calling patterns
- Model Context Protocol concepts
- Agent-to-tool contracts
- Browser and computer-use testing concepts
- External API connectors
- Permission boundaries at the interface
- Auditing action-oriented agents
- Capstone — golden tasks, mocks, evaluators, safety suite
- Capstone — CI regression run and quality report
- Presenting agent quality to a release decision

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.